DATA STRUCTURES & ALGORITHMS · C++

Searching & Sorting Solved Questions

Every problem from Part 13 of the Searching & Sorting guide, in order, with a worked C++ solution, the approach behind it, and the mistake that most often breaks it.

38	3	12	150
PROBLEMS	TIERS	PATTERNS	TESTS PASSING

Prepared for **Hari** · on-campus placement preparation
Companion to **Searching-and-Sorting-Guide.pdf** · code verified against **reference-part13-solutions.cpp**
Every solution in this document was compiled with **g++ -std=c++17 -O2 -Wall** and passes its test cases.
Generated 14 October 2026

Contents

Tier 1 — Learn the mechanics

01 LC 704 · Binary Search
02 LC 35 · Search Insert Position
03 LC 278 · First Bad Version
04 LC 69 · Sqrt(x)
05 LC 34 · Find First and Last Position of Element in Sorted Array
06 LC 367 · Valid Perfect Square
07 LC 744 · Find Smallest Letter Greater Than Target
08 LC 852 · Peak Index in a Mountain Array
09 LC 88 · Merge Sorted Array
10 LC 912 · Sort an Array
11 LC 75 · Sort Colors
12 LC 1051 · Height Checker
13 LC 977 · Squares of a Sorted Array

Tier 2 — The placement core

14 LC 33 · Search in Rotated Sorted Array
15 LC 153 · Find Minimum in Rotated Sorted Array
16 LC 162 · Find Peak Element
17 LC 74 · Search a 2D Matrix
18 LC 240 · Search a 2D Matrix II
19 LC 875 · Koko Eating Bananas
20 LC 1011 · Capacity To Ship Packages Within D Days
21 LC 540 · Single Element in a Sorted Array
22 LC 981 · Time Based Key-Value Store
23 LC 56 · Merge Intervals
24 LC 179 · Largest Number
25 LC 215 · Kth Largest Element in an Array
26 LC 1482 · Minimum Number of Days to Make m Bouquets
27 LC 274 · H-Index
28 LC 81 · Search in Rotated Sorted Array II

Tier 3 — Harder, but they do appear

29 LC 4 · Median of Two Sorted Arrays
30 LC 410 · Split Array Largest Sum
31 LC 154 · Find Minimum in Rotated Sorted Array II

32 LC 315 · Count of Smaller Numbers After Self

33 LC 327 · Count of Range Sum

34 LC 719 · Find K-th Smallest Pair Distance

35 LC 668 · Kth Smallest Number in Multiplication Table

36 LC 164 · Maximum Gap

37 LC 1095 · Find in Mountain Array

38 LC 1802 · Maximum Value at a Given Index in a Bounded Array

Tier 1 — Learn the mechanics

Thirteen problems: exact search, lower and upper bound, first-true and last-true predicates, peaks, merging, partitioning and counting sort. When you can write 704, 34 and 75 cold and say what lo and hi mean, you have the mechanics.

01 LC 704 · Binary Search

EASY Pattern 1 — Exact binary search

PROBLEM

Return the index of `target` in a sorted array of distinct integers, or −1.

APPROACH

Closed interval `[lo, hi]`: the answer, if it exists, is always between `lo` and `hi` inclusive. Compare with the middle; equal → done; smaller → the answer is strictly right of `mid`; larger → strictly left. The range empties when `lo > hi`.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int lo = 0, hi = (int)nums.size() - 1;      // search space: [lo, hi], closed
        while (lo <= hi) {                          // non-empty while lo <= hi
            int mid = lo + (hi - lo) / 2;           // no overflow
            if (nums[mid] == target) return mid;
            if (nums[mid] < target) lo = mid + 1;   // answer is strictly right of mid
            else hi = mid - 1;                      // answer is strictly left of mid
        }
        return -1;
    }
};
```

WATCH OUT

`while (lo <= hi)` goes with `hi = n − 1` and `hi = mid − 1`. Mixing in `hi = mid` loops forever on a two-element range. Compute `mid = lo + (hi − lo) / 2` out of habit — it costs nothing and never overflows.

02 LC 35 · Search Insert Position

EASY Pattern 2 — Lower bound

PROBLEM

Return the index of `target`, or where it would be inserted to keep the array sorted.

APPROACH

This is exactly **lower_bound**: the first index with `a[i] >= target`. Half-open `[lo, hi)` with `hi = n`, so "after everything" is a legal answer. If `a[mid] < target`, mid cannot be the answer — discard it; otherwise it might be — keep it with `hi = mid`.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        // lower_bound: the first index with nums[i] >= target
        int lo = 0, hi = nums.size();               // half-open [lo, hi); n is a valid answer
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] < target) lo = mid + 1;   // mid is too small: discard it
            else hi = mid;                          // mid might be the answer: keep it
        }
        return lo;                                  // lo == hi: the boundary
    }
};
```

WATCH OUT

Starting with `hi = n - 1` makes it impossible to return n when the target is larger than everything. The STL one-liner is `lower_bound(a.begin(), a.end(), t) - a.begin()` — know it, and know how to write it.

03 LC 278 · First Bad Version

EASY Pattern 3 — First true

PROBLEM

Versions 1…n; from some version on, all are bad. Find the first bad one using as few calls to `isBadVersion` as possible.

APPROACH

`isBadVersion` is a predicate that goes false, …, false, true, …, true. Find the **first true**: if `mid` is bad, the answer is `mid` or earlier (`hi = mid`); if good, strictly later (`lo = mid + 1`). The loop ends when `lo == hi`.

Complexity O(log n) calls · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int firstBadVersion(int n) {
        int lo = 1, hi = n;                          // the answer is in [lo, hi]
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;            // (lo + hi) / 2 overflows near 2^31
            if (isBadVersion(mid)) hi = mid;         // mid is bad: first bad is mid or earlier
            else lo = mid + 1;                       // mid is good: first bad is later
        }
        return lo;
    }
};
```

WATCH OUT

n can be $2^{31} - 1$, so `(lo + hi) / 2` overflows — this problem exists to catch it. Use `lo + (hi - lo) / 2`. And there is no need for `lo <= hi` plus a separate answer variable: the half-open template lands exactly on the boundary.

04 LC 69 · Sqrt(x)

EASY Pattern 3 — Last true, round up

PROBLEM

Return the integer square root of x (rounded down), without `sqrt` or `pow`.

APPROACH

Find the **last** r with `r · r ≤ x` — a true-then-false predicate. When the predicate holds, the answer is `mid` or larger, so `lo = mid`. Because a branch assigns `lo = mid`, **round mid up**.

Complexity O(log x) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int mySqrt(int x) {
        // the LAST r with r*r <= x
        long long lo = 0, hi = x;
        while (lo < hi) {
            long long mid = lo + (hi - lo + 1) / 2;  // round UP: lo = mid must make progress
            if (mid * mid <= x) lo = mid;            // mid works: answer is mid or larger
            else hi = mid - 1;
        }
        return (int)lo;
    }
};
```

WATCH OUT

With round-down `mid`, `lo = mid` loops forever once `hi = lo + 1`. And `mid * mid` overflows `int` for x near 2^{31} — compute in `long long`. Newton's method converges faster; binary search is the expected answer.

05 LC 34 · Find First and Last Position of Element in Sorted Array

MEDIUM Pattern 2 — Lower and upper bound

PROBLEM

Return the first and last index of `target` in a sorted array, or `[−1, −1]`, in O(log n).

APPROACH

One helper does both: `lowerBound(x)` = first index with `a[i] ≥ x`. The first occurrence is `lowerBound(target)` — if it is in range and equals the target. The last occurrence is `lowerBound(target + 1) − 1`: one before the first strictly bigger value.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
    // first index with a[i] >= x   (x+1 gives "first index > x")
    int lowerBound(const vector<int>& a, int x) {
        int lo = 0, hi = a.size();
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] < x) lo = mid + 1; else hi = mid;
        }
        return lo;
    }
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int first = lowerBound(nums, target);
        if (first == (int)nums.size() || nums[first] != target) return {-1, -1};
        int last = lowerBound(nums, target + 1) - 1;   // one before the first bigger value
        return {first, last};
    }
};
```

WATCH OUT

Finding one occurrence and then walking left and right is O(n) when the array is all one value — exactly what the O(log n) requirement forbids. Check `first < n` before reading `nums[first]`. `target + 1` overflows at `INT_MAX`; an explicit `upperBound` avoids that.

06 LC 367 · Valid Perfect Square

EASY Pattern 1 — Exact search, long long

PROBLEM

Is `num` a perfect square? No `sqrt` allowed.

APPROACH

Exact binary search for an r with `r · r == num` over `[1, num]`.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool isPerfectSquare(int num) {
        long long lo = 1, hi = num;
        while (lo <= hi) {
            long long mid = lo + (hi - lo) / 2;
            long long sq = mid * mid;                // long long: 46341^2 > INT_MAX
            if (sq == num) return true;
            if (sq < num) lo = mid + 1; else hi = mid - 1;
        }
        return false;
    }
};
```

WATCH OUT

`mid * mid` with `int mid` overflows once mid passes 46 340 — use `long long`. A neat O(√n) alternative: subtract 1, 3, 5, 7, ... — perfect squares are sums of consecutive odd numbers.

07 LC 744 · Find Smallest Letter Greater Than Target

EASY Pattern 2 — Upper bound with wrap-around

PROBLEM

Letters are sorted (with repeats). Return the smallest letter strictly greater than `target`, wrapping to the first letter if none is greater.

APPROACH

upper_bound: the first index with `letters[i] > target` — the same template as lower_bound with `<=` in the discard test. If it equals n, wrap to index 0: `letters[lo % n]`.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    char nextGreatestLetter(vector<char>& letters, char target) {
        // upper_bound: the first letter strictly greater than target
        int lo = 0, hi = letters.size();
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (letters[mid] <= target) lo = mid + 1; else hi = mid;
        }
        return letters[lo % letters.size()];        // none greater: wrap to the first
    }
};
```

WATCH OUT

Using `<` instead of `<=` gives lower_bound and returns `target` itself when it is present. The wrap is the whole twist of the problem; `% n` handles it without a branch.

08 LC 852 · Peak Index in a Mountain Array

MEDIUM Pattern 4 — Slope search

PROBLEM

The array strictly increases then strictly decreases. Return the index of the peak in O(log n).

APPROACH

The predicate `a[i] > a[i+1]` is false while climbing and true while descending; the peak is the **first true**. Compare `a[mid]` with `a[mid + 1]`: going up → `lo = mid + 1`; going down → `hi = mid`.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& a) {
        int lo = 0, hi = (int)a.size() - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;            // mid < hi, so a[mid+1] exists
            if (a[mid] < a[mid + 1]) lo = mid + 1;   // still climbing: peak is to the right
            else hi = mid;                           // descending: peak is mid or left
        }
        return lo;
    }
};
```

WATCH OUT

`hi` starts at `n − 1` and the loop is `lo < hi`, so `mid < hi` and `a[mid + 1]` always exists. Starting with `hi = n` reads past the end. Comparing with `a[mid − 1]` instead works too, but then round mid up.

09 LC 88 · Merge Sorted Array

EASY Pattern 12 — Merge from the back

PROBLEM

Merge sorted `b` (n elements) into sorted `a`, which has m elements followed by n empty slots.

APPROACH

Fill `a` from the **back**: the largest remaining element of either array goes into the last free slot. Writing from the back never overwrites an unread element of `a`. Stop when `b` is exhausted — whatever is left of `a` is already in place.

Complexity O(m + n) time · O(1) extra space

C++ SOLUTION

```cpp
class Solution {
public:
    void merge(vector<int>& a, int m, vector<int>& b, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;     // fill from the BACK:
        while (j >= 0) {                             // nothing is overwritten unread
            if (i >= 0 && a[i] > b[j]) a[k--] = a[i--];
            else a[k--] = b[j--];
        }                                            // leftovers of a are already in place
    }
};
```

WATCH OUT

Merging from the front overwrites `a`'s own elements before they are read. The loop runs while `j >= 0` (b remains), not while `i >= 0` — leftover `b` elements must still be copied, leftover `a` elements need nothing.

10 LC 912 · Sort an Array

MEDIUM Pattern 9 — Randomised three-way quicksort

PROBLEM

Sort an array in O(n log n) without the library sort.

APPROACH

Quicksort with a **random pivot** and a **three-way partition**: `< p`, `== p`, `> p`. Recurse only on the outer two blocks — the equal block is already in its final position. The random pivot defeats sorted input; the three-way split defeats arrays full of duplicates.

Complexity O(n log n) expected · O(log n) expected stack

C++ SOLUTION

```cpp
class Solution {
    // randomised quicksort with a THREE-WAY partition:
    //   a[lo..lt-1] < p,   a[lt..gt] == p,   a[gt+1..hi] > p
    void quickSort(vector<int>& a, int lo, int hi) {
        if (lo >= hi) return;
        int p = a[lo + rand() % (hi - lo + 1)];      // random pivot: no sorted-input worst case
        int lt = lo, i = lo, gt = hi;
        while (i <= gt) {
            if (a[i] < p) swap(a[lt++], a[i++]);
            else if (a[i] > p) swap(a[i], a[gt--]);  // do NOT advance i: new a[i] is unseen
            else i++;
        }
        quickSort(a, lo, lt - 1);                    // the == p block is already placed
        quickSort(a, gt + 1, hi);
    }
public:
    vector<int> sortArray(vector<int>& nums) {
        quickSort(nums, 0, (int)nums.size() - 1);
        return nums;
    }
};
```

WATCH OUT

Plain Lomuto quicksort with a fixed pivot TLEs here: LeetCode includes sorted arrays and arrays of one repeated value, both O(n²). Merge sort (Topic 05) is the guaranteed alternative. After swapping with `gt`, do **not** advance `i`.

11 LC 75 · Sort Colors

MEDIUM Pattern 9 — Dutch national flag

PROBLEM

Sort an array of 0s, 1s and 2s in place, in one pass.

APPROACH

Three regions grow as `i` scans: `[0, lo)` are 0s, `[lo, i)` are 1s, `(hi, n−1]` are 2s, and `[i, hi]` is unknown. A 0 swaps to `lo` (both advance), a 2 swaps to `hi` (only `hi` moves), a 1 just advances `i`.

Complexity O(n) time, one pass · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    void sortColors(vector<int>& a) {
        // Dutch national flag: [0, lo) = 0s, [lo, i) = 1s, (hi, n-1] = 2s, [i, hi] unknown
        int lo = 0, i = 0, hi = (int)a.size() - 1;
        while (i <= hi) {
            if (a[i] == 0) swap(a[lo++], a[i++]);
            else if (a[i] == 2) swap(a[i], a[hi--]);   // i stays: the swapped-in value is new
            else i++;
        }
    }
};
```

WATCH OUT

After swapping a 2 to the right, the value now at `i` came from the unknown region — it must be examined, so `i` stays. After swapping a 0 to the left, the value arriving at `i` is a known 1, so `i` advances. Counting sort (two passes) is the simpler fallback.

12 LC 1051 · Height Checker

EASY Pattern 11 — Counting sort

PROBLEM

Count the positions where `heights` differs from its sorted version.

APPROACH

Heights lie in 1…100, so count them and walk the counts to produce the sorted order on the fly, comparing as you go — no comparison sort needed.

Complexity O(n + 100) time · O(100) space

C++ SOLUTION

```cpp
class Solution {
public:
    int heightChecker(vector<int>& heights) {
        int cnt[101] = {0};                          // counting sort: heights are 1..100
        for (int h : heights) cnt[h]++;
        int wrong = 0, h = 1;
        for (int x : heights) {
            while (cnt[h] == 0) h++;                 // next value in sorted order
            if (x != h) wrong++;
            cnt[h]--;
        }
        return wrong;
    }
};
```

WATCH OUT

Copying and calling `sort` is O(n log n) and perfectly acceptable — but whenever the value range is tiny, an interviewer expects you to notice counting sort.

13 LC 977 · Squares of a Sorted Array

EASY Pattern 12 — Two pointers from the ends

PROBLEM

Given a sorted array (possibly with negatives), return the squares in sorted order.

APPROACH

Squaring breaks the order only for negatives, and the **largest** square is always at one of the two ends. Two pointers at the ends; place the bigger square at the back of the result and move that pointer inward.

Complexity O(n) time · O(n) for the output

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> sortedSquares(vector<int>& a) {
        int n = a.size(), l = 0, r = n - 1;
        vector<int> res(n);
        for (int k = n - 1; k >= 0; k--) {           // the LARGEST square is at an end
            if (abs(a[l]) > abs(a[r])) { res[k] = a[l] * a[l]; l++; }
            else { res[k] = a[r] * a[r]; r--; }
        }
        return res;
    }
};
```

WATCH OUT

Squaring and sorting is O(n log n) — the expected follow-up is O(n). Filling the result from the **front** would need the smallest square, which is somewhere in the middle; fill from the back.

Tier 2 — The placement core

These are what actually get asked. Rotated arrays, binary search on the answer, Merge Intervals and quickselect appear in nearly every on-campus drive.

14 LC 33 · Search in Rotated Sorted Array

MEDIUM Pattern 5 — One half is always sorted

PROBLEM

A sorted array of distinct values was rotated at an unknown pivot. Find `target` in O(log n).

APPROACH

At every `mid`, at least one half is sorted: if `a[lo] <= a[mid]`, the left half is; otherwise the right half is. Check whether the target lies inside the **sorted** half's range — that is the one comparison you can trust. If yes, search there; if not, search the other half.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int search(vector<int>& a, int target) {
        int lo = 0, hi = (int)a.size() - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == target) return mid;
            if (a[lo] <= a[mid]) {                      // LEFT half [lo, mid] is sorted
                if (a[lo] <= target && target < a[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else {                                    // RIGHT half [mid, hi] is sorted
                if (a[mid] < target && target <= a[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        return -1;
    }
};
```

WATCH OUT

`a[lo] <= a[mid]` needs `<=`, not `<`: when `lo == mid` (two elements left) the left "half" is a single sorted element. The range checks are half-open on the `mid` side (`target < a[mid]`), because `mid` was already compared.

15 LC 153 · Find Minimum in Rotated Sorted Array

MEDIUM Pattern 5 — Compare with the right end

PROBLEM

Return the minimum of a rotated sorted array of distinct values, in O(log n).

APPROACH

Compare `a[mid]` with `a[hi]`. If `a[mid] > a[hi]`, the drop (and the minimum) is strictly to the right. Otherwise `[mid, hi]` is sorted and the minimum is `mid` or to its left. First-true template; the answer is `a[lo]`.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int findMin(vector<int>& a) {
        int lo = 0, hi = (int)a.size() - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] > a[hi]) lo = mid + 1;    // the drop is right of mid
            else hi = mid;                       // [mid, hi] is sorted: min is mid or left
        }
        return a[lo];
    }
};
```

WATCH OUT

Comparing with `a[lo]` instead of `a[hi]` fails on an unrotated array: `a[lo] <= a[mid]` is true everywhere, and you move right past the minimum. The index of the minimum is also the **rotation count** — a common GfG question.

16 LC 162 · Find Peak Element

MEDIUM Pattern 4 — Walk uphill

PROBLEM

Return the index of any element strictly greater than its neighbours, with `a[−1] = a[n] = −∞`, in O(log n).

APPROACH

If `a[mid] < a[mid + 1]` the slope goes up, and climbing must reach a peak on the right — at worst the boundary, since `a[n] = −∞`. Otherwise a peak is at `mid` or to its left. Each step keeps a half that is guaranteed to contain a peak.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int findPeakElement(vector<int>& a) {
        int lo = 0, hi = (int)a.size() - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            // walking uphill always reaches a peak, because a[-1] = a[n] = -infinity
            if (a[mid] < a[mid + 1]) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }
};
```

WATCH OUT

It finds **a** peak, not the highest one — there is no O(log n) way to find the global maximum of an unsorted array. Neighbours are guaranteed unequal; with equal neighbours the argument breaks.

17 LC 74 · Search a 2D Matrix

MEDIUM Pattern 6 — Flatten the index

PROBLEM

Each row is sorted and starts after the previous row ends. Is `target` in the matrix?

APPROACH

Read row by row, the matrix is **one** sorted array of R·C values. Binary search on the flat index k and convert with `m[k / C][k % C]`.

Complexity O(log(R·C)) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& m, int target) {
        int R = m.size(), C = m[0].size();
        int lo = 0, hi = R * C - 1;                 // treat the matrix as ONE sorted array
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            int v = m[mid / C][mid % C];            // index -> (row, col)
            if (v == target) return true;
            if (v < target) lo = mid + 1; else hi = mid - 1;
        }
        return false;
    }
};
```

WATCH OUT

Divide by the number of **columns** for both the row and the column. Two binary searches (find the row, then search it) are also O(log R + log C) and equally good. The top-right staircase from LC 240 works too, but in O(R + C).

18 LC 240 · Search a 2D Matrix II

MEDIUM Pattern 6 — Top-right staircase

PROBLEM

Rows and columns are each sorted (but rows do not continue each other). Is `target` present?

APPROACH

Start at the **top-right** corner. Everything below is larger and everything to the left is smaller. If the value is too big, the whole column below is too big — move left. If too small, the whole row to the left is too small — move down. Each step discards a row or a column.

> **Complexity** O(R + C) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& m, int target) {
        // staircase from the TOP-RIGHT: left is smaller, down is larger
        int r = 0, c = (int)m[0].size() - 1;
        while (r < (int)m.size() && c >= 0) {
            if (m[r][c] == target) return true;
            if (m[r][c] > target) c--;              // whole column below is bigger: drop it
            else r++;                               // whole row to the left is smaller: drop it
        }
        return false;
    }
};
```

> **WATCH OUT**
>
> Starting at the top-left fails: both moves increase the value, so a too-small value gives no direction. The bottom-left corner works symmetrically. Binary searching every row is O(R log C) — fine, but not the expected answer.

19 LC 875 · Koko Eating Bananas

MEDIUM Pattern 7 — Binary search on the answer: speed

PROBLEM

Koko eats at most k bananas per hour from one pile. Find the minimum k that finishes all piles within h hours.

APPROACH

If speed k is enough, any faster speed is too — feasibility is monotone in k. Search k over `[1, max pile]`. The check: hours needed = `Σ ceil(pile / k)`, computed as `(p + k − 1) / k`. Minimise → first feasible speed.

Complexity O(n log(max pile)) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
    long long hoursAt(const vector<int>& piles, int speed) {
        long long h = 0;
        for (int p : piles) h += (p + speed - 1) / speed;   // ceil(p / speed)
        return h;
    }
public:
    int minEatingSpeed(vector<int>& piles, int h) {
        // binary search on the ANSWER: "speed k is fast enough" is monotone in k
        int lo = 1, hi = *max_element(piles.begin(), piles.end());
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (hoursAt(piles, mid) <= h) hi = mid;      // fast enough: try slower
            else lo = mid + 1;
        }
        return lo;
    }
};
```

WATCH OUT

The hours sum can exceed `int` (10^4 piles of 10^9 at speed 1) — sum in `long long`. The upper bound is the largest pile: faster than that never helps, since a pile takes at least an hour anyway.

20 LC 1011 · Capacity To Ship Packages Within D Days

MEDIUM Pattern 7 — Binary search on the answer: capacity

PROBLEM

Packages ship in order. Find the least ship capacity that delivers all of them within `days` days.

APPROACH

Feasibility is monotone in capacity. The check is greedy: load packages in order, starting a new day whenever the next one would exceed the capacity. Bounds: `lo` = the heaviest package (anything less cannot carry it), `hi` = the total (one day).

Complexity O(n log(total)) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
    int daysNeeded(const vector<int>& w, int cap) {
        int days = 1, load = 0;
        for (int x : w) {
            if (load + x > cap) { days++; load = 0; }    // greedy: start a new day
            load += x;
        }
        return days;
    }
public:
    int shipWithinDays(vector<int>& weights, int days) {
        int lo = *max_element(weights.begin(), weights.end());   // must carry the heaviest
        int hi = accumulate(weights.begin(), weights.end(), 0);  // everything in one day
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (daysNeeded(weights, mid) <= days) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
};
```

WATCH OUT

The lower bound is the **heaviest package**: no smaller capacity can carry it, and the greedy check quietly assumes every single package fits — keep it inside the range where that is true. This is LC 410 / book allocation in different words.

21 LC 540 · Single Element in a Sorted Array

MEDIUM Pattern 3 — Pair parity

PROBLEM

Every element appears twice except one. Find the single one in O(log n) time and O(1) space.

APPROACH

Before the single element, pairs start at **even** indices; after it, at odd ones. Snap `mid` to the even start of its pair. If `a[mid] == a[mid + 1]`, the pair is intact and the single element is to the right (`lo = mid + 2`); otherwise it is at `mid` or to the left.

Complexity O(log n) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int singleNonDuplicate(vector<int>& a) {
        // before the single element pairs start at EVEN indices; after it, at odd ones
        int lo = 0, hi = (int)a.size() - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (mid % 2 == 1) mid--;                // look at the pair starting at even mid
            if (a[mid] == a[mid + 1]) lo = mid + 2;  // pair intact: single is to the right
            else hi = mid;                           // pair broken: single is mid or left
        }
        return a[lo];
    }
};
```

WATCH OUT

XOR of everything is O(n) — correct, but not what is asked. Forgetting to snap `mid` to even makes the pair comparison meaningless half the time.

22 LC 981 · Time Based Key-Value Store

MEDIUM Pattern 2 — Upper bound on timestamps

PROBLEM

`set(key, value, t)` stores a value; `get(key, t)` returns the value with the largest timestamp ≤ t, or "".

APPROACH

Timestamps for each key arrive increasing, so each key's list is already sorted. `get` is "last entry with time ≤ t": find `upper_bound(t)` — the first entry with time > t — and step back one.

Complexity `set` O(1) · `get` O(log n) · O(n) space

C++ SOLUTION

```cpp
class TimeMap {
    // timestamps arrive strictly increasing, so each list is already sorted
    unordered_map<string, vector<pair<int, string>>> data;
public:
    void set(string key, string value, int timestamp) {
        data[key].push_back({timestamp, value});
    }
    string get(string key, int timestamp) {
        auto it = data.find(key);
        if (it == data.end()) return "";
        auto& v = it->second;
        // first entry with time > timestamp; the one before it is the answer
        auto ub = upper_bound(v.begin(), v.end(), timestamp,
                              [](int t, const pair<int, string>& e) { return t < e.first; });
        return ub == v.begin() ? "" : prev(ub)->second;
    }
};
```

WATCH OUT

`upper_bound` with a lambda on pairs takes `(value, element)` in that order: `[](int t, const pair<…>& e) { return t < e.first; }`. If `upper_bound` returns `begin()`, nothing is early enough — return "".

23 LC 56 · Merge Intervals

MEDIUM Pattern 12 — Sort by start, then sweep

PROBLEM

Merge all overlapping intervals.

APPROACH

Sort by start. Sweep, keeping the last merged interval: if the next one starts at or before its end, extend the end to the **max** of the two ends; otherwise start a new merged interval.

Complexity O(n log n) time · O(n) for the output

C++ SOLUTION

```cpp
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& iv) {
        sort(iv.begin(), iv.end());                  // by start, then end
        vector<vector<int>> res;
        for (auto& cur : iv) {
            if (!res.empty() && cur[0] <= res.back()[1])
                res.back()[1] = max(res.back()[1], cur[1]);   // overlap: extend
            else
                res.push_back(cur);                           // gap: new interval
        }
        return res;
    }
};
```

WATCH OUT

`[1,4],[2,3]` must stay `[1,4]` — take the `max` of the ends, not the new end. Touching intervals (`[1,4],[4,5]`) merge because the test is `<=`. Sorting `vector<vector<int>>` with no comparator already sorts by start.

24 LC 179 · Largest Number

MEDIUM Pattern 12 — Concatenation comparator

PROBLEM

Arrange non-negative integers to form the largest number; return it as a string.

APPROACH

Put a before b when the concatenation `a + b` beats `b + a` as strings. `"9" + "34"` > `"34" + "9"`, so 9 comes first. This ordering is transitive, so `sort` is safe with it. Join the result.

Complexity O(n log n · L) time for numbers of up to L digits · O(n · L) space

C++ SOLUTION

```cpp
class Solution {
public:
    string largestNumber(vector<int>& nums) {
        vector<string> s;
        for (int x : nums) s.push_back(to_string(x));
        // a goes first iff a+b > b+a  (a strict weak ordering, so sort is safe)
        sort(s.begin(), s.end(), [](const string& a, const string& b) { return a + b > b + a; });
        if (s[0] == "0") return "0";                // all zeros
        string res;
        for (auto& x : s) res += x;
        return res;
    }
};
```

WATCH OUT

Sorting the strings in plain descending order fails on `[3, 30]`: `"30" > "3"` but `"330"` beats `"303"`. If the first string is `"0"`, every number is zero — return `"0"`, not `"000"`.

25 LC 215 · Kth Largest Element in an Array

MEDIUM Pattern 9 — Quickselect

PROBLEM

Return the k-th largest element, in better than O(n log n) on average.

APPROACH

The k-th largest is at index `n − k` in ascending order. Partition around a random pivot into `< p | == p | > p`; if the target index falls in the equal block, the pivot is the answer; otherwise continue in **one** side only. n + n/2 + n/4 + ... = O(n) expected.

Complexity O(n) expected, O(n²) worst · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int findKthLargest(vector<int>& a, int k) {
        int target = (int)a.size() - k;              // its index in ascending order
        int lo = 0, hi = (int)a.size() - 1;
        while (true) {                               // QUICKSELECT: recurse into one side only
            int p = a[lo + rand() % (hi - lo + 1)];
            int lt = lo, i = lo, gt = hi;            // three-way partition around p
            while (i <= gt) {
                if (a[i] < p) swap(a[lt++], a[i++]);
                else if (a[i] > p) swap(a[i], a[gt--]);
                else i++;
            }
            if (target < lt) hi = lt - 1;            // answer is in the < p block
            else if (target > gt) lo = gt + 1;       // answer is in the > p block
            else return p;                           // inside the == p block
        }
    }
};
```

WATCH OUT

A fixed pivot makes sorted input O(n²). The three-way partition is what keeps an array of identical values linear. The heap solution (Topic 07, O(n log k)) is the safe alternative when the worst case matters; `nth_element` is the library quickselect.

26 LC 1482 · Minimum Number of Days to Make m Bouquets

MEDIUM Pattern 7 — Binary search on the answer: day

PROBLEM

Flower i blooms on `bloomDay[i]`. A bouquet needs k **adjacent** bloomed flowers. Find the earliest day to make m bouquets, or −1.

APPROACH

Waiting longer never hurts, so feasibility is monotone in the day. The check walks the flowers counting runs of bloomed ones and cuts a bouquet every k. Search the day between the earliest and latest bloom.

Complexity O(n log(max day)) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
    int bouquetsBy(const vector<int>& bloom, int day, int k) {
        int made = 0, run = 0;
        for (int b : bloom) {
            run = (b <= day) ? run + 1 : 0;          // adjacent bloomed flowers
            if (run == k) { made++; run = 0; }
        }
        return made;
    }
public:
    int minDays(vector<int>& bloomDay, int m, int k) {
        if ((long long)m * k > (long long)bloomDay.size()) return -1;   // never enough flowers
        int lo = *min_element(bloomDay.begin(), bloomDay.end());
        int hi = *max_element(bloomDay.begin(), bloomDay.end());
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (bouquetsBy(bloomDay, mid, k) >= m) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
};
```

WATCH OUT

Check `m · k > n` first — then it is impossible on any day — and compute it in `long long` (both can be 10^6). The run counter must reset to 0 at an unbloomed flower **and** after each bouquet.

27 LC 274 · H-Index

MEDIUM Pattern 11 — Counting sort

PROBLEM

Return the largest h such that the researcher has at least h papers with at least h citations each.

APPROACH

h can never exceed n, so cap citations at n and count them. Walk h from n down, accumulating how many papers have at least h citations; the first h where that count reaches h is the answer.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int hIndex(vector<int>& citations) {
        int n = citations.size();
        vector<int> cnt(n + 1, 0);                  // counting sort, capped at n
        for (int c : citations) cnt[min(c, n)]++;
        int atLeast = 0;                            // papers with >= h citations
        for (int h = n; h >= 0; h--) {
            atLeast += cnt[h];
            if (atLeast >= h) return h;             // the largest h that works
        }
        return 0;
    }
};
```

WATCH OUT

The sort-based version — sort descending, find the last i with `c[i] >= i + 1` — is O(n log n) and fine. The counting version shows you noticed the range is bounded by n. Off-by-one between 0- and 1-indexed h is the usual slip.

28 LC 81 · Search in Rotated Sorted Array II

MEDIUM Pattern 5 — Duplicates allowed

PROBLEM

LC 33 with duplicates: does `target` occur?

APPROACH

The LC 33 logic, plus one case: when `a[lo] == a[mid] == a[hi]`, you cannot tell which half is sorted (`[1,0,1,1,1]` vs `[1,1,1,0,1]`). Discard both ends — they are not the target, since `a[mid]` is not — and continue.

Complexity O(log n) typical, **O(n) worst** (all equal but one) · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool search(vector<int>& a, int target) {
        int lo = 0, hi = (int)a.size() - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == target) return true;
            if (a[lo] == a[mid] && a[mid] == a[hi]) { lo++; hi--; continue; }  // can't tell: shrink
            if (a[lo] <= a[mid]) {
                if (a[lo] <= target && target < a[mid]) hi = mid - 1; else lo = mid + 1;
            } else {
                if (a[mid] < target && target <= a[hi]) lo = mid + 1; else hi = mid - 1;
            }
        }
        return false;
    }
};
```

WATCH OUT

The worst case is genuinely O(n) and no algorithm can avoid it — be ready to explain why. Shrinking only when *all three* are equal is enough; shrinking more often throws away information.

Tier 3 — Harder, but they do appear

Binary search on splits and on counts, merge sort that counts, the pigeonhole principle, and the duplicate cases that turn O(log n) into O(n).

29 LC 4 · Median of Two Sorted Arrays

HARD Pattern 7 — Binary search the split

PROBLEM

Return the median of two sorted arrays in O(log(m + n)).

APPROACH

Split both arrays so the left parts hold half of all elements: i from the shorter array `a`, `j = half − i` from `b`. The split is right when `a[i−1] ≤ b[j]` and `b[j−1] ≤ a[i]`. If `a[i−1] > b[j]`, i is too big; else too small. Binary search i over the shorter array, with ±∞ beyond the ends.

Complexity O(log(min(m, n))) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    double findMedianSortedArrays(vector<int>& a, vector<int>& b) {
        if (a.size() > b.size()) return findMedianSortedArrays(b, a);  // search the SHORTER
        int m = a.size(), n = b.size(), half = (m + n + 1) / 2;
        int lo = 0, hi = m;                          // i = how many of a go to the left half
        while (lo <= hi) {
            int i = lo + (hi - lo) / 2, j = half - i;
            int aL = i ? a[i - 1] : INT_MIN, aR = i < m ? a[i] : INT_MAX;
            int bL = j ? b[j - 1] : INT_MIN, bR = j < n ? b[j] : INT_MAX;
            if (aL <= bR && bL <= aR) {              // a valid split: left <= right
                if ((m + n) % 2) return max(aL, bL);
                return (max(aL, bL) + (double)min(aR, bR)) / 2.0;
            }
            if (aL > bR) hi = i - 1;                 // took too many from a
            else lo = i + 1;                         // took too few from a
        }
        return 0.0;                                  // unreachable for valid input
    }
};
```

WATCH OUT

Search the **shorter** array, or `j` can go negative. The sentinels (`INT_MIN` / `INT_MAX` when i or j is 0 or at the end) are what make empty arrays and extreme splits work. Average in `double` — two `int`s near 2^{31} overflow.

30 LC 410 · Split Array Largest Sum

HARD Pattern 7 — Minimise the maximum (book allocation)

PROBLEM

Split the array into k non-empty contiguous parts to minimise the largest part sum.

APPROACH

"Largest sum ≤ cap is achievable with at most k parts" is monotone in cap. The greedy check packs elements into a part until the next one would overflow, then opens a new part. Bounds: the largest element … the total sum.

Complexity O(n log(total)) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
    int piecesNeeded(const vector<int>& a, long long cap) {   // greedy: fill each piece
        int pieces = 1;
        long long sum = 0;
        for (int x : a) {
            if (sum + x > cap) { pieces++; sum = 0; }
            sum += x;
        }
        return pieces;
    }
public:
    int splitArray(vector<int>& nums, int k) {
        // minimise the maximum: "largest sum <= cap is achievable" is monotone in cap
        long long lo = *max_element(nums.begin(), nums.end());
        long long hi = accumulate(nums.begin(), nums.end(), 0LL);
        while (lo < hi) {
            long long mid = lo + (hi - lo) / 2;
            if (piecesNeeded(nums, mid) <= k) hi = mid;
            else lo = mid + 1;
        }
        return (int)lo;
    }
};
```

WATCH OUT

This is GfG's **allocate minimum pages** and **painter's partition** — know all three names. At most k parts is enough: with fewer, any part can be split further without increasing the maximum. Sums need `long long`. There is also an O(k · n²) DP, which this beats.

31 LC 154 · Find Minimum in Rotated Sorted Array II

HARD Pattern 5 — Duplicates, O(n) worst case

PROBLEM

LC 153 with duplicates allowed.

APPROACH

Compare `a[mid]` with `a[hi]`: greater → minimum to the right; smaller → minimum at `mid` or left; **equal** → you cannot tell, but `a[hi]` has a twin at `mid`, so dropping `hi` cannot lose the minimum's value: `hi--`.

Complexity O(log n) typical, O(n) worst · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int findMin(vector<int>& a) {
        int lo = 0, hi = (int)a.size() - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] > a[hi]) lo = mid + 1;
            else if (a[mid] < a[hi]) hi = mid;
            else hi--;              // equal: cannot tell which side; a[hi] has a twin at mid
        }
        return a[lo];
    }
};
```

WATCH OUT

`hi--` is safe only because `a[mid] == a[hi]` — the value survives at `mid`. Dropping `lo` instead is not safe. Expect "*how do duplicates change the complexity?*" — worst case O(n), e.g. `[1,1,1,0,1,1,1]`.

32 LC 315 · Count of Smaller Numbers After Self

HARD Pattern 10 — Merge sort on indices

PROBLEM

For every i, count the elements to its right that are smaller than `nums[i]`.

APPROACH

Merge sort an array of **indices** by value, so each count stays attached to its original position. When merging, a left element is placed after all right elements already taken that are smaller than it — each of those was to its right in the original array. Add that count to it.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
    vector<int> cnt, idx, tmp;
    void sortCount(const vector<int>& a, int lo, int hi) {   // sorts idx[lo..hi] by value
        if (lo >= hi) return;
        int mid = lo + (hi - lo) / 2;
        sortCount(a, lo, mid);
        sortCount(a, mid + 1, hi);
        int i = lo, j = mid + 1, k = lo;
        while (i <= mid) {
            // every right-half element already taken is smaller AND was to the right
            while (j <= hi && a[idx[j]] < a[idx[i]]) tmp[k++] = idx[j++];
            cnt[idx[i]] += j - (mid + 1);
            tmp[k++] = idx[i++];
        }
        while (j <= hi) tmp[k++] = idx[j++];
        for (k = lo; k <= hi; k++) idx[k] = tmp[k];
    }
public:
    vector<int> countSmaller(vector<int>& nums) {
        int n = nums.size();
        cnt.assign(n, 0); tmp.assign(n, 0); idx.resize(n);
        iota(idx.begin(), idx.end(), 0);              // sort INDICES, so answers stay attached
        sortCount(nums, 0, n - 1);
        return cnt;
    }
};
```

WATCH OUT

Sorting values directly loses which count belongs to which position — sort indices. Equal values are not "smaller", so take right elements only while strictly less. A Fenwick tree over compressed values is the other standard O(n log n) solution.

33 LC 327 · Count of Range Sum

HARD Pattern 10 — Merge sort on prefix sums

PROBLEM

Count subarrays whose sum lies in `[lower, upper]`.

APPROACH

A range sum is `P[j] − P[i]` for prefix sums with i < j. Merge sort the prefix array. For each left prefix `P[i]`, the right half is sorted, so the valid `P[j]` form a contiguous window — two monotone pointers find its bounds. Then merge.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
    vector<long long> tmp;
    // counts pairs i < j in P[lo..hi] with lower <= P[j] - P[i] <= upper, and sorts P
    int sortCount(vector<long long>& P, int lo, int hi, int lower, int upper) {
        if (lo >= hi) return 0;
        int mid = lo + (hi - lo) / 2;
        int res = sortCount(P, lo, mid, lower, upper) + sortCount(P, mid + 1, hi, lower, upper);
        int a = mid + 1, b = mid + 1;                // a window in the sorted right half
        for (int i = lo; i <= mid; i++) {
            while (a <= hi && P[a] - P[i] < lower) a++;
            while (b <= hi && P[b] - P[i] <= upper) b++;
            res += b - a;                            // right-half sums in [lower, upper]
        }
        merge(P.begin() + lo, P.begin() + mid + 1, P.begin() + mid + 1,
              P.begin() + hi + 1, tmp.begin() + lo);
        copy(tmp.begin() + lo, tmp.begin() + hi + 1, P.begin() + lo);
        return res;
    }
public:
    int countRangeSum(vector<int>& nums, int lower, int upper) {
        int n = nums.size();
        vector<long long> P(n + 1, 0);               // prefix sums; range sum = P[j] - P[i]
        for (int i = 0; i < n; i++) P[i + 1] = P[i] + nums[i];
        tmp.assign(n + 1, 0);
        return sortCount(P, 0, n, lower, upper);
    }
};
```

WATCH OUT

Prefix sums need `long long` — the test with `INT_MAX` and `INT_MIN` checks it. Include the empty prefix `P[0] = 0`, or subarrays starting at index 0 are missed. Both window pointers only move forward as `P[i]` grows, which is what keeps the count linear.

34 LC 719 · Find K-th Smallest Pair Distance

HARD Pattern 8 — Count pairs within d

PROBLEM

Return the k-th smallest distance `|a[i] − a[j]|` among all pairs.

APPROACH

There are n² pairs — too many to list. Sort, then binary search on the distance d: the answer is the smallest d with at least k pairs ≤ d. Counting pairs ≤ d is a two-pointer pass over the sorted array.

Complexity O(n log n + n log W) for value range W · O(1) extra space

C++ SOLUTION

```cpp
class Solution {
    long long pairsWithin(const vector<int>& a, int d) {   // pairs with distance <= d
        long long count = 0;
        for (int i = 0, j = 0; i < (int)a.size(); i++) {
            while (a[i] - a[j] > d) j++;             // two pointers on the sorted array
            count += i - j;
        }
        return count;
    }
public:
    int smallestDistancePair(vector<int>& nums, int k) {
        sort(nums.begin(), nums.end());
        int lo = 0, hi = nums.back() - nums.front();
        while (lo < hi) {                            // the smallest d with >= k pairs within d
            int mid = lo + (hi - lo) / 2;
            if (pairsWithin(nums, mid) >= k) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
};
```

WATCH OUT

Sort first — the two-pointer count needs it. The answer is always a real pair distance: the smallest d whose count reaches k is hit exactly at one. Building all n² distances is O(n² log n) and runs out of memory.

35 LC 668 · Kth Smallest Number in Multiplication Table

HARD Pattern 8 — Count by formula

PROBLEM

Return the k-th smallest entry of the m × n multiplication table.

APPROACH

Binary search on the value x. Row i contains `i, 2i, …, ni`, so it has `min(x / i, n)` entries ≤ x — summing over rows counts the whole table in O(m). The answer is the smallest x whose count reaches k.

Complexity O(m log(m·n)) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int findKthNumber(int m, int n, int k) {
        int lo = 1, hi = m * n;
        while (lo < hi) {                           // smallest x with >= k entries <= x
            int mid = lo + (hi - lo) / 2;
            long long count = 0;
            for (int i = 1; i <= m; i++) count += min(mid / i, n);   // row i: i, 2i, ..., ni
            if (count >= k) hi = mid; else lo = mid + 1;
        }
        return lo;                                  // always a value in the table
    }
};
```

WATCH OUT

m · n up to 9·10⁸ fits in `int`, but the count sum is safer in `long long`. The table has m·n entries — building it is out of the question; the whole point is counting without building.

36 LC 164 · Maximum Gap

MEDIUM Pattern 11 — Buckets and the pigeonhole principle

PROBLEM

Return the largest difference between successive elements in sorted order, in linear time.

APPROACH

n values span `max − min`, so the largest gap is at least `(max − min) / (n − 1)`. Use buckets of that width (at least 1): no gap inside a bucket can be the maximum. Keep only each bucket's min and max, then compare each non-empty bucket's min with the previous non-empty bucket's max.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int maximumGap(vector<int>& a) {
        int n = a.size();
        if (n < 2) return 0;
        int mn = *min_element(a.begin(), a.end()), mx = *max_element(a.begin(), a.end());
        if (mn == mx) return 0;
        // n-1 gaps share (mx - mn): the max gap >= ceil((mx-mn)/(n-1)) = bucket width,
        // so it is never INSIDE a bucket — only between consecutive non-empty buckets
        int width = max(1, (mx - mn) / (n - 1));
        int nb = (mx - mn) / width + 1;
        vector<int> bMin(nb, INT_MAX), bMax(nb, INT_MIN);
        for (int x : a) {
            int b = (x - mn) / width;
            bMin[b] = min(bMin[b], x);
            bMax[b] = max(bMax[b], x);
        }
        int best = 0, prevMax = mn;
        for (int b = 0; b < nb; b++) {
            if (bMin[b] == INT_MAX) continue;        // empty bucket
            best = max(best, bMin[b] - prevMax);
            prevMax = bMax[b];
        }
        return best;
    }
};
```

WATCH OUT

Sorting is O(n log n) and passes, but the problem asks for linear. Guard `max == min` (all equal) and `n < 2`. Radix sort followed by one scan is the other linear solution.

37 LC 1095 · Find in Mountain Array

HARD Pattern 4 — Peak, then two searches

PROBLEM

Return the minimum index of `target` in a mountain array accessible only through `get(i)`, using at most 100 calls.

APPROACH

Three binary searches: find the peak by the slope (LC 852); search the ascending side normally; if not found, search the descending side with the comparisons **reversed**. Search the left side first, because the smaller index must win.

Complexity O(log n) calls — about 3 $\log_2 n \approx 45$ for $n = 10^4$ · O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int findInMountainArray(int target, MountainArray& m) {
        int n = m.length();
        int lo = 0, hi = n - 1;                      // 1. find the peak
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (m.get(mid) < m.get(mid + 1)) lo = mid + 1; else hi = mid;
        }
        int peak = lo;
        lo = 0, hi = peak;                           // 2. ascending side: the SMALLER index wins
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2, v = m.get(mid);
            if (v == target) return mid;
            if (v < target) lo = mid + 1; else hi = mid - 1;
        }
        lo = peak + 1, hi = n - 1;                   // 3. descending side: comparisons flip
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2, v = m.get(mid);
            if (v == target) return mid;
            if (v > target) lo = mid + 1; else hi = mid - 1;
        }
        return -1;
    }
};
```

WATCH OUT

On the descending side, `v > target` means move **right**. Each slope step makes two `get` calls; caching the values helps if the limit is tight. Checking the right side first returns a larger index when the target appears on both sides.

38 LC 1802 · Maximum Value at a Given Index in a Bounded Array

MEDIUM Pattern 7 — Maximise, with arithmetic sums

PROBLEM

Build an array of n positive integers, neighbours differing by at most 1, total at most `maxSum`. Maximise the value at `index`.

APPROACH

Binary search the peak value v (maximise → last true, round up). The cheapest array with peak v descends by 1 on each side down to 1, then stays at 1. Each side's sum is an arithmetic series, computed in O(1).

Complexity O(log maxSum) time · O(1) space

C++ SOLUTION

```cpp
class Solution {
    // minimum total of `len` cells descending by 1 from just below `top`, floor 1
    long long side(long long top, long long len) {
        if (top - 1 >= len)                          // top-1, top-2, ..., top-len
            return (top - 1 + top - len) * len / 2;
        long long sloped = top - 1;                  // top-1, ..., 1, then 1s
        return sloped * (sloped + 1) / 2 + (len - sloped);
    }
public:
    int maxValue(int n, int index, int maxSum) {
        int lo = 1, hi = maxSum;
        while (lo < hi) {                            // the LARGEST peak that fits
            int mid = lo + (hi - lo + 1) / 2;        // round up: lo = mid must progress
            long long total = mid + side(mid, index) + side(mid, n - 1 - index);
            if (total <= maxSum) lo = mid; else hi = mid - 1;
        }
        return lo;
    }
};
```

WATCH OUT

Each side has two cases — the slope reaches 1 inside the side, or it does not — and mixing them up is the usual bug. Use `long long`: v up to 10^9 squared overflows. `lo = mid` means rounding mid up.